
AI Assisted Coding

Assignment – 11.4

Name : Akula Vivek

Roll Number : 2403A51258

Batch : 11

Task 1: Implementing a Stack (LIFO)

- Task: Use AI to help implement a Stack class in Python with the following operations: push(), pop(), peek(), and is_empty().
- Instructions:
 - o Ask AI to generate code skeleton with docstrings.
 - o Test stack operations using sample data.
 - o Request AI to suggest optimizations or alternative implementations (e.g., using collections.deque).
- Expected Output:
 - o A working Stack class with proper methods, Google-style docstrings, and inline comments for tricky parts.

Code :

```

# Create a new stack
my_stack = Stack()

# Check if the stack is empty
print(f"Is the stack empty? {my_stack.is_empty()}")

# Push some items onto the stack
my_stack.push(10)
my_stack.push(20)
my_stack.push(30)

# Check if the stack is empty after pushing
print(f"Is the stack empty? {my_stack.is_empty()}")

# Peek at the top item
print(f"Top item: {my_stack.peek()}")

# Pop items from the stack
print(f"Popped item: {my_stack.pop()}")
print(f"Popped item: {my_stack.pop()}")

# Peek at the top item after popping
print(f"Top item after popping: {my_stack.peek()}")

# Pop the last item
print(f"Popped item: {my_stack.pop()}")

# Check if the stack is empty again
print(f"Is the stack empty? {my_stack.is_empty()}")

# Try to pop from an empty stack (will raise an error)
try:
    my_stack.pop()
except IndexError as e:
    print(f"Error: {e}")

```

Output:

```

➡ Is the stack empty? True
Is the stack empty? False
Top item: 30
Popped item: 30
Popped item: 20
Top item after popping: 10
Popped item: 10
Is the stack empty? True
Error: pop from empty stack

```

Explanation:

1. `my_stack = Stack()`: This line creates a new, empty instance of your `Stack` class and assigns it to the variable `my_stack`.

2. `print(f"Is the stack empty? {my_stack.is_empty()}")`: This calls the `is_empty()` method on the `my_stack` object to check if the stack is currently empty. Since it was just created, it will print `True`.
3. `my_stack.push(10), my_stack.push(20), my_stack.push(30)`: These lines use the `push()` method to add the integers 10, 20, and 30 onto the top of the stack, in that order.
4. `print(f"Is the stack empty? {my_stack.is_empty()}")`: This checks if the stack is empty again after adding items. It will now print `False`.
5. `print(f"Top item: {my_stack.peek()}")`: This calls the `peek()` method, which returns the item at the top of the stack (which is 30) without removing it.
6. `print(f"Popped item: {my_stack.pop()}")`, `print(f"Popped item: {my_stack.pop()}")`: These lines call the `pop()` method twice. Each call removes and returns the item from the top of the stack. First 30 is popped, then 20.
7. `print(f"Top item after popping: {my_stack.peek()}")`: After the two pops, the item now at the top of the stack is 10. This line calls `peek()` again to show this.
8. `print(f"Popped item: {my_stack.pop()}")`: This pops the last remaining item (10) from the stack.
9. `print(f"Is the stack empty? {my_stack.is_empty()}")`: This checks the stack's empty status again. Since all items have been popped, it will print `True`.
10. `try...except IndexError as e: print(f"Error: {e}")`: This block attempts to call `my_stack.pop()` one more time. Since the stack is now empty, calling `pop()` will raise an `IndexError`. The `try...except` block catches this specific error, and the code inside the `except` block is executed, printing the error message "pop from empty stack". This demonstrates how the `pop()` method handles an empty stack.

Task 2: Queue Implementation with Performance Review

- Task: Implement a Queue with `enqueue()`, `dequeue()`, and `is_empty()` methods.
- Instructions:
 - o First, implement using Python lists.
 - o Then, ask AI to review performance and suggest a more efficient implementation (using `collections.deque`).
- Expected Output:
 - o Two versions of a queue: one with lists and one optimized with `deque`, plus an AI-generated performance comparison.

Code:

```

# Code for ListQueue implementation
class ListQueue:
    """
    A simple Queue implementation using a Python list.

    A queue is a linear data structure that follows the FIFO (First-In, First-Out) principle.
    """

    def __init__(self):
        """
        Initializes an empty Queue.
        """
        self._items = []

    def enqueue(self, item):
        """
        Adds an item to the rear of the queue.

        Args:
            item: The item to be added to the queue.
        """
        self._items.append(item)

    def dequeue(self):
        """
        Removes and returns the item from the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if not self.is_empty():
            return self._items.pop(0) # Removing from the front of a list is O(n)

```

```

        return self._items.pop(0) # Removing from the front of a list is O(1)
    else:
        raise IndexError("dequeue from empty queue")

def is_empty(self):
    """
    Checks if the queue is empty.

    Returns:
        True if the queue is empty, False otherwise.
    """
    return len(self._items) == 0

def peek(self):
    """
    Returns the item at the front of the queue without removing it.

    Returns:
        The item at the front of the queue.

    Raises:
        IndexError: If the queue is empty.
    """
    if not self.is_empty():
        return self._items[0]
    else:
        raise IndexError("peek from empty queue")

```

```

▶ # Test the list-based Queue implementation
print("--- Testing ListQueue ---")
my_list_queue = ListQueue()
print(f"List Queue - Is empty? {my_list_queue.is_empty()}")
my_list_queue.enqueue(10)
my_list_queue.enqueue(20)
my_list_queue.enqueue(30)
print(f"List Queue - Is empty? {my_list_queue.is_empty()}")
print(f"List Queue - Front item: {my_list_queue.peek()}")
print(f"List Queue - Dequeued item: {my_list_queue.dequeue()}")
print(f"List Queue - Dequeued item: {my_list_queue.dequeue()}")
print(f"List Queue - Front item after dequeuing: {my_list_queue.peek()}")
print(f"List Queue - Dequeued item: {my_list_queue.dequeue()}")
print(f"List Queue - Is empty? {my_list_queue.is_empty()}")
try:
    my_list_queue.dequeue()
except IndexError as e:
    print(f"List Queue - Error: {e}")
print("-" * 25)

# Code for DequeueQueue implementation
from collections import deque

class DequeueQueue:
    """
    A Queue implementation using collections.deque.
    """
    def __init__(self):
        """
        Initializes an empty DequeueQueue.
        """
        self._items = deque()

    def enqueue(self, item):
        """
        Adds an item to the rear of the queue.

```

```

    Args:
        item: The item to be added to the queue.
    """
    self._items.append(item) # append to the right is O(1)

def dequeue(self):
    """
    Removes and returns the item from the front of the queue.

    Returns:
        The item at the front of the queue.

    Raises:
        IndexError: If the queue is empty.
    """
    if not self.is_empty():
        return self._items.popleft() # pop from the left is O(1)
    else:
        raise IndexError("dequeue from empty queue")

def is_empty(self):
    """
    Checks if the queue is empty.

    Returns:
        True if the queue is empty, False otherwise.
    """
    return len(self._items) == 0

def peek(self):
    """
    Returns the item at the front of the queue without removing it.

    Returns:
        The item at the front of the queue.

```



```
Returns:
    The item at the front of the queue.

Raises:
    IndexError: If the queue is empty.
"""
if not self.is_empty():
    return self._items[0]
else:
    raise IndexError("peek from empty queue")

# Test the collections.deque-based Queue implementation
print("--- Testing DequeueQueue ---")
my_dequeue_queue = DequeueQueue()
print(f"Deque Queue - Is empty? {my_dequeue_queue.is_empty()}")
my_dequeue_queue.enqueue(100)
my_dequeue_queue.enqueue(200)
my_dequeue_queue.enqueue(300)
print(f"Deque Queue - Is empty? {my_dequeue_queue.is_empty()}")
print(f"Deque Queue - Front item: {my_dequeue_queue.peek()}")
print(f"Deque Queue - Dequeued item: {my_dequeue_queue.dequeue()}")
print(f"Deque Queue - Dequeued item: {my_dequeue_queue.dequeue()}")
print(f"Deque Queue - Front item after dequeuing: {my_dequeue_queue.peek()}")
print(f"Deque Queue - Dequeued item: {my_dequeue_queue.dequeue()}")
print(f"Deque Queue - Is empty? {my_dequeue_queue.is_empty()}")
try:
    my_dequeue_queue.dequeue()
except IndexError as e:
    print(f"Deque Queue - Error: {e}")
print("-" * 25)
```


Output :

```
➡ --- Testing ListQueue ---
List Queue - Is empty? True
List Queue - Is empty? False
List Queue - Front item: 10
List Queue - Dequeued item: 10
List Queue - Dequeued item: 20
List Queue - Front item after dequeuing: 30
List Queue - Dequeued item: 30
List Queue - Is empty? True
List Queue - Error: dequeue from empty queue
-----

--- Testing DequeueQueue ---
Deque Queue - Is empty? True
Deque Queue - Is empty? False
Deque Queue - Front item: 100
Deque Queue - Dequeued item: 100
Deque Queue - Dequeued item: 200
Deque Queue - Front item after dequeuing: 300
Deque Queue - Dequeued item: 300
Deque Queue - Is empty? True
Deque Queue - Error: dequeue from empty queue
-----
```

Task 3: Singly Linked List with Traversal

- Task: Implement a Singly Linked List with operations: insert_at_end(), delete_value(), and traverse().
- Instructions:
 - o Start with a simple class-based implementation (Node, LinkedList).
 - o Use AI to generate inline comments explaining pointer updates (which are non-trivial).
 - o Ask AI to suggest test cases to validate all operations.
- Expected Output:
 - o A functional linked list implementation with clear comments explaining the logic of insertions and deletions

Code:

```
# Create an instance of the LinkedList class
my_list = LinkedList()

# Insert several elements into the list
print("Inserting elements:")
my_list.insert_at_end(10)
my_list.insert_at_end(20)
my_list.insert_at_end(30)
my_list.insert_at_end(40)

# Traverse and print the list to show the initial state
print("Initial list:")
my_list.traverse()

# Delete a value from the middle of the list
print("\nDeleting value 30:")
my_list.delete_value(30)

# Traverse and print the list again to show the effect of the deletion
print("List after deleting 30:")
my_list.traverse()

# Delete the head of the list
print("\nDeleting head (value 10):")
my_list.delete_value(10)

# Traverse and print the list to show the updated list
print("List after deleting 10:")
my_list.traverse()

# Attempt to delete a value that does not exist in the list
print("\nAttempting to delete non-existent value 50:")
my_list.delete_value(50)

# Traverse and print the list to confirm no changes were made
print("List after attempting to delete 50:")
my_list.traverse()
```

```

# Traverse and print the list to confirm no changes were made
print("List after attempting to delete 50:")
my_list.traverse()

# Insert another element and then delete the only remaining element in the list
print("\nInserting 50 and deleting 20 and 40 and 50:")
my_list.insert_at_end(50)
my_list.delete_value(20)
my_list.delete_value(40)
my_list.delete_value(50)

# Traverse and print the empty list
print("List after deleting all elements:")
my_list.traverse()

# Attempt to delete from an empty list
print("\nAttempting to delete from an empty list:")
my_list.delete_value(100)

```

Output:

```

➞ Inserting elements:
Initial list:
10 -> 20 -> 30 -> 40 -> None

Deleting value 30:
List after deleting 30:
10 -> 20 -> 40 -> None

Deleting head (value 10):
List after deleting 10:
20 -> 40 -> None

Attempting to delete non-existent value 50:
Value 50 not found in the list.
List after attempting to delete 50:
20 -> 40 -> None

Inserting 50 and deleting 20 and 40 and 50:
List after deleting all elements:
None

Attempting to delete from an empty list:
List is empty. Cannot delete.

```

Explanation :

1. `my_list = LinkedList()`: This line creates a new, empty instance of your `LinkedList` class.
2. `print("Inserting elements:")`: This simply prints a header to indicate the start of insertion tests.
3. `my_list.insert_at_end(10)`, `my_list.insert_at_end(20)`, `my_list.insert_at_end(30)`, `my_list.insert_at_end(40)`: These lines call the `insert_at_end()` method to add the values 10, 20, 30, and 40 to the end of the linked list. The list will become 10 -> 20 -> 30 -> 40 -> None.
4. `print("Initial list:")`: Prints a header.
5. `my_list.traverse()`: Calls the `traverse()` method to print the current elements of the list. This confirms the initial insertions.
6. `print("\nDeleting value 30:")`: Prints a header for the deletion test.
7. `my_list.delete_value(30)`: Calls `delete_value()` to remove the node with the value 30 from the list. The list should become 10 -> 20 -> 40 -> None.
8. `print("List after deleting 30:")`: Prints a header.
9. `my_list.traverse()`: Calls `traverse()` to show the list after deleting 30.
10. `print("\nDeleting head (value 10:")`: Prints a header for deleting the head node.
11. `my_list.delete_value(10)`: Calls `delete_value()` to remove the node with the value 10, which is currently the head. The list should become 20 -> 40 -> None.
12. `print("List after deleting 10:")`: Prints a header.
13. `my_list.traverse()`: Calls `traverse()` to show the list after deleting the head.
14. `print("\nAttempting to delete non-existent value 50:")`: Prints a header for testing deletion of a non-existent value.
15. `my_list.delete_value(50)`: Calls `delete_value()` to try and remove 50, which is not in the list. The `delete_value` method should handle this gracefully (e.g., by printing a message) and the list should remain unchanged: 20 -> 40 -> None.
16. `print("List after attempting to delete 50:")`: Prints a header.
17. `my_list.traverse()`: Calls `traverse()` to confirm the list is unchanged.
18. `print("\nInserting 50 and deleting 20 and 40 and 50:")`: Prints a header for testing deletion of multiple elements, including inserting a new one first.
19. `my_list.insert_at_end(50)`: Inserts 50 at the end: 20 -> 40 -> 50 -> None.

- 20. `my_list.delete_value(20)`, `my_list.delete_value(40)`, `my_list.delete_value(50)`: These lines sequentially delete the remaining elements (20, then 40, then 50) until the list is empty.
- 21. `print("List after deleting all elements:")`: Prints a header.
- 22. `my_list.traverse()`: Calls `traverse()` on the now empty list, which should just print `None`.
- 23. `print("\nAttempting to delete from an empty list:")`: Prints a header for testing deletion from an empty list.
- 24. `my_list.delete_value(100)`: Calls `delete_value()` on the empty list. The method should handle this by printing a message like "List is empty. Cannot delete."

Task 4: Binary Search Tree (BST)

- Task: Implement a Binary Search Tree with methods for insert(), search(), and inorder_traversal().
- Instructions:
 - o Provide AI with a partially written Node and BST class.
 - o Ask AI to complete missing methods and add docstrings.
 - o Test with a list of integers and compare outputs of search() for present vs absent elements.
- Expected Output:
 - o A BST class with clean implementation, meaningful docstrings, and correct traversal output.

Code:

```
class Node:
    """
    Represents a node in a Binary Search Tree.
    """
    def __init__(self, key):
        """
        Initializes a new Node with a given key.

        Args:
            key: The value to be stored in the node.
        """
        self.key = key
        self.left = None
        self.right = None

class BST:
    """
    Represents a Binary Search Tree.
    """
    def __init__(self):
        """
        Initializes an empty Binary Search Tree.
        """
        self.root = None

    def insert(self, key):
        """
        Inserts a new node with the given key into the BST.

        Args:
            key: The value to be inserted.
        """
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursive(self.root, key)
```



```
def _insert_recursive(self, node, key):
    """
    Helper function for recursive insertion.

    Args:
        node: The current node being considered.
        key: The value to be inserted.
    """
    if key < node.key:
        if node.left is None:
            node.left = Node(key)
        else:
            self._insert_recursive(node.left, key)
    else: # key >= node.key (allowing duplicates to the right)
        if node.right is None:
            node.right = Node(key)
        else:
            self._insert_recursive(node.right, key)

def search(self, key):
    """
    Searches for a node with the given key in the BST.

    Args:
        key: The value to search for.

    Returns:
        True if the key is found, False otherwise.
    """
    return self._search_recursive(self.root, key)

def _search_recursive(self, node, key):
    """
    Helper function for recursive search.
```



```
    Args:
        node: The current node being considered.
        key: The value to search for.

    Returns:
        True if the key is found, False otherwise.
    """
    if node is None:
        return False
    if node.key == key:
        return True
    elif key < node.key:
        return self._search_recursive(node.left, key)
    else:
        return self._search_recursive(node.right, key)

def inorder_traversal(self):
    """
    Performs an in-order traversal of the BST and returns a list of keys.

    Returns:
        A list containing the keys in ascending order.
    """
    result = []
    self._inorder_recursive(self.root, result)
    return result

def _inorder_recursive(self, node, result):
    """
    Helper function for recursive in-order traversal.

    Args:
        node: The current node being considered.
        result: The list to store the traversal results.
    """
    if node:
        self._inorder_recursive(node.left, result)
        result.append(node.key)
```



```

    if node:
        self._inorder_recursive(node.left, result)
        result.append(node.key)
        self._inorder_recursive(node.right, result)

# Test the BST implementation
print("--- Testing Binary Search Tree ---")

# Create a new BST
bst = BST()

# List of integers to insert
elements_to_insert = [50, 30, 70, 20, 40, 60, 80, 35]

# Insert elements into the BST
print("Inserting elements:", elements_to_insert)
for element in elements_to_insert:
    bst.insert(element)

# Perform in-order traversal
print("In-order traversal:", bst.inorder_traversal())

# Test search for present elements
print("\nTesting search for present elements:")
present_elements = [20, 40, 70, 35]
for element in present_elements:
    found = bst.search(element)
    print(f"Searching for {element}: {found}")

# Test search for absent elements
print("\nTesting search for absent elements:")
absent_elements = [10, 55, 90, 45]
for element in absent_elements:
    found = bst.search(element)
    print(f"Searching for {element}: {found}")

print("-" * 30)

```

Output:

```
➞ --- Testing Binary Search Tree ---  
Inserting elements: [50, 30, 70, 20, 40, 60, 80, 35]  
In-order traversal: [20, 30, 35, 40, 50, 60, 70, 80]  
  
Testing search for present elements:  
Searching for 20: True  
Searching for 40: True  
Searching for 70: True  
Searching for 35: True  
  
Testing search for absent elements:  
Searching for 10: False  
Searching for 55: False  
Searching for 90: False  
Searching for 45: False  
-----
```

Task 5: Graph Representation and BFS/DFS Traversal

- Task: Implement a Graph using an adjacency list, with traversal methods BFS() and DFS().
- Instructions:
 - o Start with an adjacency list dictionary.
 - o Ask AI to generate BFS and DFS implementations with inline comments.
 - o Compare recursive vs iterative DFS if suggested by AI.
- Expected Output:
 - o A graph implementation with BFS and DFS traversal methods, with AI-generated comments explaining traversal steps.

Code:

```
from collections import deque

class Graph:
    """
    Represents a graph using an adjacency list.
    """
    def __init__(self):
        # Initialize an empty dictionary to store the adjacency list
        self.adj_list = {}

    def add_edge(self, u, v):
        """
        Adds an edge between nodes u and v (for an undirected graph).

        Args:
            u: The first node.
            v: The second node.
        """
        # Add edge from u to v
        if u not in self.adj_list:
            self.adj_list[u] = []
        self.adj_list[u].append(v)

        # Add edge from v to u (for undirected graph)
        if v not in self.adj_list:
            self.adj_list[v] = []
        self.adj_list[v].append(u)

    def bfs(self, start_node):
        """
        Performs a Breadth-First Search (BFS) traversal starting from a given node.

        Args:
            start_node: The node to start the traversal from.
        """
        print(f"BFS traversal starting from node {start_node}:", end=" ")
        # Initialize a queue for BFS and add the start node
        queue = deque([start_node])
```

```

# Continue as long as the queue is not empty
while queue:
    # Dequeue a node from the front of the queue
    current_node = queue.popleft()
    # Process the current node (e.g., print it)
    print(current_node, end=" ")

    # Iterate through the neighbors of the current node
    # Use .get() with a default empty list to handle nodes with no neighbors
    for neighbor in self.adj_list.get(current_node, []):
        # If the neighbor has not been visited
        if neighbor not in visited:
            # Mark the neighbor as visited
            visited.add(neighbor)
            # Enqueue the neighbor
            queue.append(neighbor)
    print() # Print a newline at the end of traversal

def dfs_iterative(self, start_node):
    """
    Performs an iterative Depth-First Search (DFS) traversal starting from a given node using a stack.

    Args:
        start_node: The node to start the traversal from.
    """
    print(f"DFS iterative traversal starting from node {start_node}:", end=" ")
    # Initialize a stack for DFS and add the start node
    stack = [start_node]
    # Initialize a set to keep track of visited nodes
    visited = set()

    # Continue as long as the stack is not empty
    while stack:
        # Pop a node from the top of the stack
        current_node = stack.pop()

        # If the current node has not been visited
        if current_node not in visited:

```

```

        # Mark the current node as visited
        visited.add(current_node)

        # Iterate through the neighbors of the current node
        # Reverse the neighbors to maintain a consistent exploration order (e.g., smaller nodes first if adjacent)
        for neighbor in reversed(self.adj_list.get(current_node, [])):
            # If the neighbor has not been visited
            if neighbor not in visited:
                # Push the neighbor onto the stack
                stack.append(neighbor)
        print() # Print a newline at the end of traversal

    def _dfs_recursive_helper(self, node, visited):
        """
        Helper function for recursive Depth-First Search.

        Args:
            node: The current node being considered.
            visited: A set to keep track of visited nodes.
        """
        # Mark the current node as visited
        visited.add(node)
        # Process the current node (e.g., print it)
        print(node, end=" ")

        # Iterate through the neighbors of the current node
        for neighbor in self.adj_list.get(node, []):
            # If the neighbor has not been visited
            if neighbor not in visited:
                # Recursively call the helper function for the neighbor
                self._dfs_recursive_helper(neighbor, visited)

    def dfs_recursive(self, start_node):
        """
        Performs a recursive Depth-First Search (DFS) traversal starting from a given node.

```

```

        print(f"DFS recursive traversal starting from node {start_node}:", end=" ")
        # Initialize an empty set to keep track of visited nodes
        visited = set()
        # Call the recursive helper function starting from the start node
        self._dfs_recursive_helper(start_node, visited)
        print() # Print a newline at the end of traversal

    # Example usage and testing:

    # Instantiate the Graph class
    g = Graph()

    # Add edges to create a sample graph
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('C', 'E')
    g.add_edge('D', 'E')
    g.add_edge('D', 'F')
    g.add_edge('E', 'F')

    # Display the adjacency list (optional, for verification)
    # print("Adjacency List:", g.adj_list)

    # Define the starting node for traversals
    start_node = 'A'

    # Perform BFS traversal
    g.bfs(start_node)

    # Perform iterative DFS traversal
    g.dfs_iterative(start_node)

    # Perform recursive DFS traversal
    g.dfs_recursive(start_node)

```

Output:

```
➡ BFS traversal starting from node A: A B C D E F  
DFS iterative traversal starting from node A: A B D E C F  
DFS recursive traversal starting from node A: A B D E C F
```